⚠ Atividade 16 — Tratando os Erros

UC: Programação Back-End | **Bloco 3 — Aula de 21/10/2026**
Formato: EM GRUPO, no repositório do grupo | **Tempo:** 60 minutos (aulas 4 e 5)
Avaliação: formativa — casos de erro tratados

Divisão de trabalho

Quem	Responsável por
Integrante 1	Parte 1 + Parte 2 (validação do Livro)
Integrante 2	Parte 3 (validação da Categoria)
Integrante 3	Parte 4 (middlewares)
Integrante 4	Testes das Partes 2 a 4 no Postman + Parte 5

Se o grupo tiver menos de 4 integrantes, acumulem tarefas.

Parte 1 — Quem errou? (5 min)

Escrevam **400**, **404** ou **500**. A primeira já está feita.

Situação	Status
Cadastrar um livro sem título	400
Buscar o livro 99999	404
Cadastrar um livro com preço `-10`	400
O programador esqueceu uma linha e o código quebrou	500
`GET /livross` (com dois "s")	404
Cadastrar uma categoria com nome `"ab"`	

Parte 2 — Validação do Livro (15 min)

Sigam o **guia, seções 2.1, 2.2 e 2.3**.

Testem no Postman:

#	Requisição	Corpo	Esperado	Obtido
1	`POST /livros`	`{ "autor": "Ninguem" }`	400	`
2	`POST /livros`	`{ "titulo": "X", "autor": "Y", "preco": -5, "estoque": 1 }`	400	`
3	`POST /livros`	`{ "titulo": "X", "autor": "Y", "preco": "abc", "estoque": 1 }`	400	`
4	`PATCH /livros/1`	`{ "estoque": -1 }`	400	`

#	Requisição	Corpo	Esperado	Obtido
5	PATCH /livros/1	{ "preco": 50 }	200	`
6	POST /livros	{ "titulo": "O Hobbit", "autor": "J.R.R. Tolkien", "preco": 39.9, "estoque": 8 }	201	`

Nos 400, a resposta deve trazer a mensagem do erro, por exemplo `{ "erro": "O titulo e obrigatorio" }`.

Parte 3 — Validação da Categoria (15 min)

3.1 — A função

No `categoriaService.js`, colem a função abaixo (antes do `criarCategoria`) e completem as 3 lacunas `____`:

```
function validarCategoria(dados, completo) {
    if (!dados) return "Envie os dados da categoria em JSON";

    if (completo && !dados.____) return "O nome e obrigatorio";

    if (
        dados.nome !== undefined &&
        (typeof dados.nome !== "string" || dados.nome.length < ____)
    ) {
        return "O nome deve ter pelo menos 3 letras";
    }

    return ____;
}
```

`??` Dicas: o campo obrigatório é o nome da categoria. O tamanho mínimo está na mensagem. E quando está tudo certo, a função devolve "nada" — vejam como termina o `validarLivro`.

Acrescentem `validarCategoria` no `module.exports`.

3.2 — No controller

No `categoriaController.js`, colem no começo de `criar`, `atualizar` e `atualizarParcial`:

```
const erro = categoriaService.validarCategoria(req.body, true);
if (erro) {
    res.status(400).json({ erro: erro });
    return;
}
```

Lembrem: no `atualizarParcial` é **`false`**.

3.3 — Testar

#	Requisição	Corpo	Esperado	Obtido
1	POST /categorias	{ "descricao": "Sem nome" }	400	
2	POST /categorias	{ "nome": "ab" }	400	
3	PATCH /categorias/1	{ "nome": "x" }	400	
4	POST /categorias	{ "nome": "Infantil", "descricao": "Livros para criancas" }	201	

Parte 4 — Middlewares de 404 e 500 (15 min)

Sigam o **guia, seções 3.1, 3.2 e 3.3**.

Testem:

#	Requisição	Esperado	Obtido
1	`GET /rota-que-nao-existe`	404, `{ "erro": "Rota nao encontrada" }`	
2	`GET /livros`	200 (nada mudou)	

O 500 de propósito:

1. No `livroController.js`, na função `listar`, colem como **primeira linha**:

```
throw new Error("Teste de erro 500");
```

2. Salvem e rodem `GET /livros` → esperado: **500**, `{ "erro": "Erro interno no servidor" }`.
3. Olhem o terminal: deve aparecer `ERRO: Teste de erro 500`.
4. **Apaguem a linha**, salvem e confiram que `GET /livros` voltou a dar **200**.

Passo	Esperado	Obtido
`GET /livros` com o `throw`	500	
`GET /livros` sem o `throw`	200	

Parte 5 — Nada quebrou? (5 min)

Rodem a coleção da **Atividade 15** no Runner (**...** → **Run collection** → **Run**).

Todos os testes passaram? __ de __

Se algum falhou, a mudança de hoje quebrou alguma coisa. Leiam o nome do teste, corrijam e rodem de novo.

Parte 6 — Enviar

```
git add .
git commit -m "feat: valida entrada e padroniza respostas de erro (400/404/500)"
git push
```

⭐ Se sobrar tempo

Hoje, `GET /livros/abc` dá 404. Mas o livro "abc" não existe, ou o pedido está errado? O certo é **400**.

1. Criem `src/middlewares/validarId.js`:

```
function validarId(req, res, next) {
  if (isNaN(Number(req.params.id))) {
    res.status(400).json({ erro: "O id deve ser um numero" });
    return;
```

```
    }
    next();
}

module.exports = validarId;
```

2. No `livroRoutes.js`, importem no topo (`const validarId = require("../middlewares/validarId");`) e coloquem o `validarId` **antes** do controller nas 4 rotas com `/:id`:

```
router.get("/:id", validarId, livroController.buscarPorId);
```

3. Testem: `GET /livros/abc` → 400. `GET /livros/1` → 200.

✅ Checklist de entrega

- [ ] Parte 1 preenchida
- [ ] Validação do Livro: os 6 testes da Parte 2 batendo
- [ ] Validação da Categoria: os 4 testes da Parte 3 batendo
- [ ] Middlewares: 404 e 500 em JSON (Parte 4)
- [ ] Linha do `throw` de teste **apagada**
- [ ] Coleção da Atividade 15 passando inteira
- [ ] `git push` feito

⁉️ Se der erro

Sintoma	Solução
Erro continua em HTML	Confiram os 4 parâmetros: `tratarErros(err, req, res, next)`
Toda rota dá "Rota nao encontrada"	Os dois `app.use` dos middlewares vão logo antes do `app.listen`
`validarLivro is not a function`	Falta no `module.exports` do `livroService.js`
`PATCH` com um campo só dá 400	No `atualizarParcial` o segundo valor é `false`
`POST` certo dá 400 no preço	Número sem aspas: `"preco": 39.9`
A coleção da Atividade 15 falhou no PUT	Confiram se o corpo do PUT tem os 4 campos